# Performance tips

## Purpose

The run time of SWAT+ depends on the length of the simulation, the number of objects (HRUs, channels, aquifers, reservoirs), the time step and the amount of output. The disk space a run needs depends almost entirely on the output switched on in `print.prt`. This page covers what can be changed.

## One run per processor core

SWAT+ is a serial program: the source has no OpenMP or MPI code, and one run uses one processor core. To use several cores, run several scenarios or calibration samples at the same time, each in a separate copy of the project folder. Output file names are fixed, so two runs in the same folder overwrite each other's files.

## Build type

The executables on the releases page are optimized builds (`Rel` in the file name; see Install). When building from source, set the build type:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

`-j 8` only speeds up the compilation. Without a build type, CMake adds no optimization flag and the executable name has no `-Rel` or `-Dbg` suffix. A `Debug` build adds run-time checks (array bounds and floating-point traps with gfortran) and runs much slower; use it to track down a crash and switch back afterwards. With gfortran on Windows, the 62.0.1 CMake setup adds the gfortran run-time checks to every build type, including `Release`. See Building from source.

## Time step

`step` in `time.sim` sets the number of time steps per day. With a value above 1, precipitation, infiltration, surface runoff and channel and reservoir routing are calculated for every time step, while plant growth, soil water and nutrient cycling still run once a day. Run time grows with the number of steps. Use a sub-daily step only when the question needs it, for example Green-Ampt infiltration or sub-daily peak flows.

## Output volume

Each `y` in the daily column of a `print.prt` row writes one row per object per day, so daily output is about 365 times the size of yearly output and 30 times the size of monthly output. Average annual output adds one row per object. Ways to reduce it:

- Switch daily output on only for the rows you will use, and use monthly, yearly or average annual output for the rest.
- Limit the daily rows to the period you need with the print period on line 3 of `print.prt` (`day_start`, `yrc_start`, `day_end`, `yrc_end` and `interval`), and skip warm-up years with `nyskip`. See Print period.
- Leave `csvout` at `n` unless you need the `.csv` files: with `y`, every output file is written twice, once as text and once as `.csv`.
- With gwflow, check the gwflow rows of `print.prt`. Several gwflow output groups are on unless a row switches them off; see gwflow outputs.

`files_out.out` lists every output file a run opened. Check it after a short first run to see what a long run will write.

### Carbon output files

The carbon files are the largest outputs, because the per-layer files have a value for every soil layer and variable.

- **Legacy carbon rows.** The `hru_cb` and `hru_cb_vars` rows with `l` in the daily column write per-layer diagnostics for every HRU on every day. The `Ames_sub1` reference dataset has this setting, and a full 46-year run of its 12 HRUs writes about 9 GB, almost all of it these files. Set the daily column of these rows to `n` unless you need the daily layer values. See Legacy CSU carbon outputs.
- **Per-layer carbon rows.** In 62.0.1 the files of the `hru_cb_lyr`, `hru_cb_cpool`, `hru_cb_npool`, `hru_cb_plt`, `hru_cb_flux`, `hru_cb_drv`, `hru_cb_dyn` and `hru_cb_snap` rows repeat their block of HRU rows once per HRU. Their size grows with the square of the number of HRUs: 100 HRUs give 100 times the rows they should. Use yearly or average annual output for these rows in models with many HRUs. See Repeated rows.
- **Number of layers.** The per-layer files have as many columns per variable as the HRU with the most soil layers. An optional `carbon_layers.prt` file sets a smaller number, for example to write only the top three layers. See Layer count file.

Check the free disk space before a long run with daily output (`df -h .` on Linux and macOS). A disk that fills during a run stops it, often without a clear message; see Runs that stop without a message.

## Object count and storage

Run time grows roughly in proportion to the number of HRUs, channels, aquifers and routing units. When testing inputs, use a short simulation period or a small part of the watershed, and run the full model once the inputs are settled.

SWAT+ reads its inputs at the start and writes the outputs during the run. With many daily outputs, a project folder on a local disk runs noticeably faster than one on a network share.

## Summary

| Change | Effect |
|---|---|
| `CMAKE_BUILD_TYPE=Release` | optimized code; a `Debug` build runs much slower |
| `step` 0 or 1 in `time.sim` | daily calculation, fastest |
| daily output only where needed | smaller files, faster runs |
| print period and `nyskip` in `print.prt` | fewer daily rows |
| `csvout = n` | half the output files |
| no `l` in the daily column of `hru_cb` and `hru_cb_vars` | avoids the largest carbon files |
| yearly or average annual per-layer carbon files | avoids the repeated HRU rows growing with the square of the HRU count |
| several runs in separate folders | uses several processor cores |

#### Sources

- Build flags: `CMakeLists.txt` and `CMakePresets.json`
- Output switches: `basin_print_codes_read.f90`, file opening in `output_landscape_init.f90`
- Carbon writers and their calls: `command.f90`, `soil_nutcarb_write.f90`, `soil_carbvar_write.f90`, layer count in `carbon_layers_read.f90`